



Simulación Monte Carlo de detección de Muones cósmicos.

Antonia Morales

¹*Departamento de Física, Universidad Técnica Federico Santa María, San Joaquín, Chile*

1. Introducción

1.1. Rayos cósmicos y muones en la atmósfera.

Los rayos cósmicos son partículas de altas energías que viajan por el espacio y constantemente chocan con la atmósfera terrestre, al hacerlo interactúan con las partículas de la atmósfera y como consecuencia se crean partículas secundarias como piones (π^+ , π^- , π^0), kaones (K), entre otros, siendo los piones los más abundantes.

Dada su breve vida media, los piones neutros decaen instantáneamente a dos fotones los cuales interactúan con la atmósfera produciendo pares electron positrón los cuales producen más fotones, así generando una gran cascada electromagnética.

Los piones cargados tienen una vida media de $\approx 2,6 \times 10^{-8}$ s y decaen mediante los procesos:

$$\pi^+ \rightarrow \mu^+ + \nu_\mu$$

y

$$\pi^- \rightarrow \mu^- + \bar{\nu}_\mu$$

apreciando muones y neutrinos muónicos los cuales ambos son leptones, no se ven afectadas por la fuerza nuclear fuerte y pueden atravesar fácilmente kilómetros de atmósfera. Los muones atmosféricos típicamente se producen con energías relativistas y viajan a una velocidad cercana a la de la luz, por lo que, si bien su vida media es de $\approx 2,2 \times 10^{-6}$ s, la mayoría sobrevive hasta llegar a la superficie de la tierra gracias a la dilatación temporal.

Cuando un muón se produce en la atmósfera, nace con una cierta energía determinada por la energía de la partícula que lo produjo y el espectro de energía de los muones se puede describir mediante:

$$\frac{dN}{dE} \propto E^{-(\gamma+1)} \quad (1)$$

con $\gamma \approx 1,7$ para E hasta $\approx 10^6$ GeV [3], es decir, el número de muones disminuye al aumentar la energía del muón, por lo que hay más muones de baja energía que de alta energía.

Sin embargo, al propagarse por la atmósfera los muones interactúan con los átomos que se encuentra en el camino transfiriéndoles energía mediante ionización y excitación. La pérdida de energía por ionización se puede describir cuantitativamente mediante el modelo de Bethe Bloch:

$$-\frac{dE}{dx} = Kz^2 \frac{Z}{A} \frac{\rho}{\beta^2} \left[\frac{1}{2} \ln \frac{2m_e c^2 \beta^2 \gamma^2 T_{\text{máx}}}{I^2} - \beta^2 \right] \quad (2)$$

donde:

- $K = 0,307 \text{ MeV} \cdot \text{cm}^2/\text{g}$
- z : carga de la partícula incidente en unidades de carga elemental e
- Z : número atómico del material absorbente
- A : masa atómica del material absorbente
- ρ : densidad del material en g/cm^3
- β : velocidad relativa de la partícula cargada
- γ : factor de Lorentz
- $m_e c^2$: energía en reposo del electrón
- T_{max} : máxima energía cinética que la partícula incidente puede transferir a un electrón libre del medio en una pura colisión;

$$T_{max} = \frac{2m_e c^2 \beta^2 \gamma^2}{1 + 2\gamma(m_e/M) + (m_e/M)^2}$$

con m_e : masa del electrón y M : masa de la partícula incidente.

- I : potencial de excitación medio del material

Por otra parte, el ángulo de incidencia también es importante, dado que a mayor ángulo cenital, mayor es la distancia que tiene que recorrer un muón para llegar a la superficie y por ende pierde más energía y solo los muones más energéticos logran ser detectados.

1.2. Fotomultiplicador y centellador

Para detectar muones es posible utilizar materiales centelladores en conjunto con un fotomultiplicador. Los materiales centelladores son aquellos que emiten luz al ser golpeados por radiación ionizante, estos se usan en conjunto con un fotomultiplicador, el cual consiste de un fotocátodo que recibe fotones y mediante efecto fotoeléctrico emite un electrón, el cual es multiplicado por los dínodos generando electrones secundarios que serán recibidos por el ánodo y convertidos en una señal eléctrica.

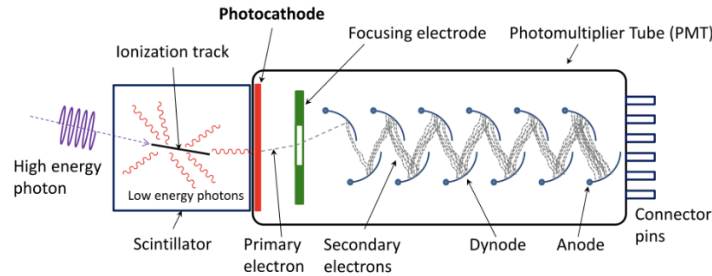


Figura 1: Esquema de funcionamiento de un fotomultiplicador con centellador.

1.3. Electrónica del detector

El fotomultiplicador recibe fotones incidentes y los transforma en fotoelectrones mediante un proceso estocástico gobernado por la eficiencia cuántica (QE), que se define como Es la proporción de fotoelectrones producidos y los fotones que realmente fueron absorbidos por el material:. Para un conjunto de fotones con tiempos de llegada $\{t_\gamma\}$, la generación de fotoelectrones se modela como un proceso de Bernoulli independiente para cada fotón, de modo que el número total de fotoelectrones sigue una estadística de tipo Poisson:

$$N_{\text{PE}} \sim \text{Poisson}(\lambda), \quad \lambda = QE \cdot N_\gamma. \quad (3)$$

Los fotoelectrones generados son posteriormente amplificados en la cadena de dinodos del PMT, produciendo una ganancia efectiva estocástica G' alrededor de una ganancia media G , con fluctuaciones asociadas al proceso de multiplicación electrónica:

$$G' \sim \mathcal{N}(G, \sigma_G^2). \quad (4)$$

El tiempo de llegada de cada fotoelectrón al ánodo se expresa como

$$t_{\text{anodo}} = t_\gamma + t_{\text{transit}} + \delta t_{\text{TTS}} + t_0, \quad \delta t_{\text{TTS}} \sim \mathcal{N}(0, \sigma_{\text{TTS}}^2), \quad (5)$$

donde t_{transit} corresponde al tiempo medio de tránsito en el tubo fotomultiplicador, σ_{TTS} representa la dispersión temporal intrínseca del dispositivo (Transit Time Spread), y t_0 corresponde a un desplazamiento temporal asociado a la geometría del detector o a la calibración del canal.

La amplitud de la señal asociada a cada fotoelectrón se modela como proporcional a la ganancia efectiva del PMT,

$$A_i = V_{PE} \cdot \frac{G'}{G}, \quad (6)$$

donde V_{PE} fija la escala de un pulso correspondiente a un único fotoelectrón.

Finalmente, la señal analógica total en el ánodo puede escribirse como la superposición de contribuciones individuales:

$$V(t) = \sum_{i=1}^{N_{\text{PE}}} A_i h(t - t_i) + \mathcal{N}(0, \sigma_n^2), \quad (7)$$

donde $h(t)$ representa la función de respuesta del sistema electrónico, típicamente aproximada por una gaussiana que incorpora los efectos del shaping del PMT y de la electrónica de front-end, y $\mathcal{N}(0, \sigma_n^2)$ modela el ruido electrónico aditivo.

2. Simulación

2.1. Generación de muones

La clase *GeneradorDeMuones* es la encargada de producir los muones que ingresan a la simulación. Para cada evento, genera aleatoriamente un ángulo cenital θ siguiendo la distribución angular esperada

para los muones cósmicos a nivel del suelo:

$$\frac{dN}{d\Omega} = \cos^2 \theta$$

así como un ángulo azimutal ϕ distribuido uniformemente entre 0 y 2π . Además, asigna a cada muón una energía inicial obtenida a partir del espectro de energía dado por (1). Finalmente, los ángulos generados se transforman en un vector de dirección tridimensional (dx,dy,dz), que define la trayectoria del muón a través de la atmósfera y del detector.

```
class GeneradorDeMuones:
    def __init__(self,
                  alpha: float = 2.7,
                  E_min_MeV: float = 5000, # 5 GeV
                  E_max_MeV: float = 1.0e6): # 1 TeV
        self.alpha = alpha
        self.E_min = E_min_MeV
        self.E_max = E_max_MeV
        self._gam = alpha - 1
        self._ratio = (E_min_MeV / E_max_MeV) ** self._gam

    def AnguloCenital(self, n: int) -> Tuple[np.ndarray, np.ndarray]:
        r = RNG.uniform(0, 1, n)
        cos_theta = np.clip((r) ** (1/3), 0, 1)
        theta = np.arccos(cos_theta)
        phi = RNG.uniform(0, 2 * np.pi, n)

        return theta, phi

    def Energia(self, n: int) -> np.ndarray:
        r = RNG.uniform(0.0, 1.0, n)

        return self.E_min * (1.0 - r * (1.0 - self._ratio)) ** (-1.0 / self._gam)

    def generate(self, n: int) -> List[dict]:
        theta, phi = self.AnguloCenital(n)
        E = self.Energia(n)

    def generar(self, n: int) -> List[dict]:
        theta, phi = self.AnguloCenital(n)
        E = self.Energia(n)

        dx = np.sin(theta) * np.cos(phi)
        dy = np.sin(theta) * np.sin(phi)
        dz = -np.cos(theta)
        return [{'theta': theta[i], 'phi': phi[i], 'E0': E[i],
                  'dir': np.array([dx[i], dy[i], dz[i]])} for i in range(n)]
```

2.2. Propagación atmosférica

El módulo de propagación atmosférica simula la propagación de un muón desde la parte superior de la atmósfera ≈ 10 km hasta el detector. En primer lugar se calcula la longitud real de la trayectoria recorrida,



que depende del ángulo cenital según $L = \frac{H}{\cos\theta}$ de modo que los muones más inclinados atraviesan una mayor cantidad de aire.

La pérdida de energía a lo largo de esta trayectoria se modela mediante la ecuación de Bethe-Bloch (2), que describe la ionización producida por una partícula cargada al atravesar un medio. La atmósfera se divide en pequeños pasos de longitud fija y, en cada uno de ellos, se calcula la pérdida de energía del muón y se resta de su energía cinética. Si durante el recorrido la energía cae por debajo de un umbral mínimo, se considera que el muón se detiene antes de alcanzar el detector y el evento es descartado. En caso contrario, el módulo devuelve la energía con la que el muón llega al nivel del detector además se implementa un método que indica la supervivencia de un muón atravesando atmósfera hasta una altura/espesor h , usando pérdida de energía por Bethe-Bloch para luego ver el flujo por altitud

```
class BetheBloch:

    def __init__(self, Z: float, A: float, I_eV: float, rho: float):
        self.Z = Z
        self.A = A
        self.I = I_eV * 1e-6
        self.rho = rho
        self.ZA = Z / A

    def cinetica(self, Ekin: float) -> Tuple[float, float, float]:

        gam = (Ekin + M_MUON) / M_MUON
        bet = np.sqrt(max(1.0 - 1.0/gam**2, 0.0))
        return bet, bet*gam, gam

    def dEdx(self, Ekin: float) -> float:

        if Ekin < 0.1:
            return 0.0
        beta, _, gamma = self.cinetica(Ekin)
        b2 = beta**2
        r = M_ELEC / M_MUON
        Tmax = (2.0 * M_ELEC * b2 * gamma**2) / (1.0 + 2.0*gamma*r + r**2)
        if Tmax <= 0.0:
            return 1.5 * self.rho
        arg = 2.0 * M_ELEC * b2 * gamma**2 * Tmax / self.I**2
        if arg <= 1.0:
            return 1.5 * self.rho
        val = K_BB * self.ZA / b2 * (0.5 * np.log(arg) - b2)
        return max(val, 1.5) * self.rho # MeV/cm

class propagacion:

    def __init__(self,
                  H_cm: float = 10e5,
                  rho_air: float = 1.225e-3,
                  n_steps: int = 500):

        self.bb = BetheBloch(Z=7.22, A=14.5, I_eV=85.7, rho=rho_air)
        self.H = H_cm
```

```
self.Nsteps = n_steps

def propagate(self, E0_MeV: float, theta_rad: float) -> Optional[float]:

    cos_t = np.cos(theta_rad)
    L      = self.H / cos_t
    dx     = L / self.Nsteps

    if cos_t <= 0:
        return None

    Ekin   = E0_MeV - M_MUON

    for _ in range(self.Nsteps):
        if Ekin < 5.0:
            return None
        Ekin -= self.bb.dEdx(Ekin) * dx
    return Ekin + M_MUON

def sobrevive_hasta_altura(self, E0_MeV, theta, h_cm):
    cos_t = np.cos(theta)
    if cos_t <= 0:
        return False

    L = h_cm / cos_t
    dx = L / self.Nsteps

    Ekin = E0_MeV - M_MUON

    for _ in range(self.Nsteps):
        if Ekin < 5.0:
            return False
        Ekin -= self.bb.dEdx(Ekin) * dx

    return True
```

2.3. Geometría del detector y métodos

Este bloque define la geometría y posición del detector, se definen dos capas de 80×80 cm de material centellador BC-408 separadas a una distancia fija. Para cada muón generado, se selecciona aleatoriamente un punto de cruce en el plano medio del detector ($z=0$) y, utilizando su vector dirección, se extrapola la trayectoria hacia las capas superior e inferior. Luego se determina si el muón atraviesa cada capa dentro de su área activa y además, se calcula la longitud efectiva recorrida dentro del material centellador, la cual depende del ángulo de incidencia y del grosor de la capa.

Cuando ocurre una intersección en ambas capas se calcula la longitud recorrida dentro del material. Finalmente se retorna un diccionario que contiene información sobre los impactos en cada capa, incluyendo si la capa fue atravesada o no, las posiciones de impacto y las longitudes de trayectoria recorridas en los centelladores.



```
@dataclass
class capa:
    name: str
    z_pos: float
    cx: float = 0.0
    cy: float = 0.0
    width_x: float = 40.0
    width_y: float = 40.0
    thickness: float = 1.0
    bb: BetheBloch = field(default_factory=lambda:
        BetheBloch(Z=5.61, A=11.0, I_eV=64.7, rho=1.032))

class Geometria:

    def __init__(self,
        separacion: float = 30.0,
        ancho: float = 80.0,
        grosor: float = 1.0):
        z_top = +separacion / 2.0
        z_bot = -separacion / 2.0

        self.top = capa('TOP', z_top, width_x=ancho,
            width_y=ancho, thickness=grosor)

        self.bot = capa('BOTTOM', z_bot, width_x=ancho,
            width_y=ancho, thickness=grosor)

        self.sep = separacion

    @staticmethod
    def impacto(pos: np.ndarray, direc: np.ndarray,
        c: capa) -> Optional[np.ndarray]:
        dz = direc[2]

        if abs(dz) < 1e-12:
            return None

        t = (c.z_pos - pos[2]) / dz

        if t < 0:
            return None

        hit = pos + t * direc
        ok = (abs(hit[0] - c.cx) <= c.width_x / 2.0 and
            abs(hit[1] - c.cy) <= c.width_y / 2.0)

        return hit if ok else None

    @staticmethod
    def track(direc: np.ndarray, thickness: float) -> float:

        cos_t = abs(direc[2])
        return thickness / max(cos_t, 1e-6)
```

```
def trace(self, muon: dict) -> dict:

    d = muon['dir'].copy()
    if d[2] < 0:
        d = -d

    x0 = RNG.uniform(-self.top.width_x / 2.0, self.top.width_x / 2.0)
    y0 = RNG.uniform(-self.top.width_y / 2.0, self.top.width_y / 2.0)
    p0 = np.array([x0, y0, 0.0])

    res = dict(hit_top=False, hit_bottom=False,
              L_top=0.0, L_bottom=0.0,
              pos_top=None, pos_bottom=None)

    d_up = np.array([-d[0], -d[1], abs(d[2])])
    hit_top = self.impacto(p0, d_up, self.top)
    if hit_top is not None:
        res.update(hit_top=True, pos_top=hit_top,
                  L_top=self.track(d, self.top.thickness))

    d_dn = np.array([d[0], d[1], -abs(d[2])])
    hit_bot = self.impacto(p0, d_dn, self.bot)
    if hit_bot is not None:
        res.update(hit_bottom=True, pos_bottom=hit_bot,
                  L_bottom=self.track(d, self.bot.thickness))

    return res
```

2.4. Deposición de energía

La clase Deposición calcula la energía depositada por un muón al atravesar una capa de centellador utilizando únicamente la pérdida de energía media dada por la ecuación de Bethe-Bloch. Como entrada recibe la energía total del muón, la longitud recorrida dentro del material y un objeto que contiene las propiedades físicas de la capa. Primero convierte la energía total en energía cinética restando la masa del muón. Si la energía cinética es demasiado pequeña o la longitud recorrida es nula, retorna una deposición de energía igual a cero. En caso contrario, calcula la pérdida media de energía por unidad de longitud (dE/dx) mediante el modelo de Bethe-Bloch y la multiplica por la distancia recorrida en el material. El valor obtenido corresponde a la energía depositada en la capa, expresada en MeV, y es retornado como resultado. A diferencia de un modelo basado en distribuciones de Landau, este enfoque no incluye fluctuaciones estadísticas evento a evento, sino que utiliza únicamente la pérdida de energía promedio esperada para el muón.

```
class Deposicion:
```



```
@staticmethod
def sample(E_total: float, L_cm: float, c: capa) -> float:

    Ekin = E_total - M_MUON

    if Ekin < 0.5 or L_cm <= 0:
        return 0.0

    dEdx_mean = c.bb.dEdx(Ekin)

    Edep = dEdx_mean * L_cm

    return float(Edep)
```

2.5. Centellador

La clase Centellador modela la producción de fotones de centelleo en una capa de plástico BC-408 después del paso de un muón. Como parámetros se utiliza un rendimiento luminoso de 10 000 fotones por MeV depositado, un tiempo de decaimiento de 2.1 ns [1] y una eficiencia de colección óptica del 5 %. A partir de la energía depositada en el material, calcula el número medio esperado de fotones que alcanzan el fotomultiplicador y genera el número real de fotones mediante una distribución de Poisson. Posteriormente, se asigna a cada fotón un tiempo de emisión obtenido de una distribución exponencial caracterizada por la constante de decaimiento del centellador. La función retorna un arreglo que contiene los tiempos de llegada de todos los fotones producidos en un evento.

```
class Centellador:
def __init__(self,
              LY: float = 10_000.0,
              tau: float = 2.1,
              eta: float = 0.05):
    self.LY = LY
    self.tau = tau
    self.eta = eta

def photon_times(self, Edep: float) -> np.ndarray:
    n = RNG.poisson(max(Edep * self.LY * self.eta, 0.0))
    if n > 0:
        return RNG.exponential(self.tau, n)
    else:
        return np.array([])
```

2.6. Electrónica de fotomultiplicador

Los tiempos de llegada de los fotones producidos en el centellador son convertidos en fotoelectrones con una probabilidad igual a la eficiencia cuántica, de modo que solo una fracción aleatoria de fotones produce fotoelectrones. Para cada fotoelectrón generado se añade un tiempo de tránsito medio o tiempo promedio que tarda un electrón dentro del PMT desde que entra hasta que llega al ánodo de 20 ns y una fluctuación temporal gaussiana asociada al Transit Time Spread (TTS). Además, cada pulso recibe una amplitud proporcional a la ganancia del PMT, la cual presenta fluctuaciones estadísticas evento a evento.

```
class PMT:

    def __init__(self,
                  QE: float = 0.25,
                  ganancia: float = 1.0e6,
                  gain_sig: float = 0.40,
                  TTS_ns: float = 0.30,
                  V_PE: float = 2.5e-3,
                  transit: float = 20.0):

        self.QE = QE
        self.G = ganancia
        self.Gsig = gain_sig * ganancia
        self.TTS = TTS_ns
        self.V_PE = V_PE
        self.transit = transit

    def detectar(self,
                 ph_times: np.ndarray,
                 t0: float = 0.0) -> Tuple[np.ndarray, np.ndarray]:

        if len(ph_times) == 0:
            return np.array([]), np.array([])
        mask = RNG.uniform(0.0, 1.0, len(ph_times)) < self.QE
        t = ph_times[mask]
        if len(t) == 0:
            return np.array([]), np.array([])
        t_an = t + self.transit + RNG.normal(0.0, self.TTS, len(t)) + t0
        g = np.abs(RNG.normal(self.G, self.Gsig, len(t)))
        amps = self.V_PE * g / self.G
        return t_an, amps
```

Posteriormente, la clase Electronica simula la respuesta de la electrónica del pmt. A partir de los tiempos y amplitudes de los fotoelectrones generados por el PMT, construye una forma de onda sumando pulsos gaussianos individuales. Sobre esta señal se añade ruido electrónico gaussiano y posteriormente se realiza una digitalización mediante un convertidor analógico-digital (ADC) de 12 bits con un rango dinámico de 0 a 2 V. La electrónica también determina el instante de disparo mediante un discriminador de umbral fijo, identificando el primer cruce de la señal sobre un nivel predefinido de 5 mV. Finalmente, se calculan parámetros característicos de la señal, como la carga integrada y la amplitud máxima del pulso.

```
class Electronica:

    def __init__(self,
                  t_start: float = 0.0,
                  t_end: float = 300.0,
                  dt: float = 0.1, # ns
                  noise_mV: float = 0.5,
                  ADC_bits: int = 12,
                  Vrange: float = 2.0, # V
```

```

        threshold_V: float = 5.0e-3,    # 5 mV
        pulse_sig: float = 1.8):        # ns    sigma del pulso PE
    self.t_ax      = np.arange(t_start, t_end, dt)
    self.dt        = dt
    self.noise     = noise_mV * 1e-3
    self.Vrange   = Vrange
    self.ADC_lev   = 2**ADC_bits
    self.thr       = threshold_V
    self.psig      = pulse_sig

    def forma(self,
              pe_times: np.ndarray,
              pe_amps: np.ndarray) -> np.ndarray:

        wf = np.zeros(len(self.t_ax))
        for tp, ap in zip(pe_times, pe_amps):

            if self.t_ax[0] - 5*self.psig < tp < self.t_ax[-1] + 5*self.psig:
                wf += ap * np.exp(-0.5 * ((self.t_ax - tp) / self.psig)**2)
        wf += RNG.normal(0.0, self.noise, len(self.t_ax))
        return wf

    def digi(self, wf: np.ndarray) -> np.ndarray:

        return (np.clip(wf, 0.0, self.Vrange) /
                self.Vrange * (self.ADC_lev - 1)).astype(np.int32)

    def threshold_tiempo(self, wf: np.ndarray) -> Optional[float]:

        above = np.where(wf >= self.thr)[0]
        if len(above) == 0:
            return None
        i = above[0]
        if i > 0 and wf[i] > wf[i-1]:
            dt_frac = (self.thr - wf[i-1]) / (wf[i] - wf[i-1])
            return self.t_ax[i-1] + dt_frac * self.dt
        return float(self.t_ax[i])

    def carga(self, wf: np.ndarray) -> float:
        return float(np.sum(np.clip(wf, 0.0, None)) * self.dt)

    def peak(self, wf: np.ndarray) -> float:
        return float(np.max(wf))

```

2.6.1. trigger por coincidencias

Este bloque de código implementa un sistema de trigger por coincidencia temporal entre dos detectores. El método aceptar recibe los tiempos de detección en el detector superior e inferior, los cuales provienen de la electrónica de lectura tras el cruce de umbral. Si alguno de los dos tiempos no está definido (por ausencia de señal), el evento se rechaza automáticamente. En caso contrario, se calcula la diferencia absoluta entre ambos tiempos y se compara con una ventana temporal fija, esto selecciona eventos correlacionados en los que una misma partícula atraviesa ambos planos del detector en un intervalo compatible con su tiempo



de vuelo, mientras que rechaza señales aleatorias, ruido electrónico y coincidencias accidentales.

```
class trigger:
    def __init__(self, window_ns: float = 5):
        self.window = window_ns

    def aceptar(self,
                t_top: Optional[float],
                t_bot: Optional[float]) -> bool:
        if t_top is None or t_bot is None:
            return False
        return abs(t_bot - t_top) <= self.window
```

2.7. Reconstrucción angular

La clase **Reconstruccion** calcula el ángulo cenital de un muón a partir del tiempo de vuelo medido entre las dos capas del detector asumiendo que los muones viajan con velocidad cercana a la de la luz. Conociendo la separación entre los centelladores y la diferencia temporal entre las señales registradas en las capas superior e inferior, se calcula el coseno del ángulo cenital mediante una relación geométrica derivada de la longitud efectiva recorrida por la partícula. Si el tiempo medido es negativo o conduce a un valor físicamente imposible para el coseno, el evento se descarta, en caso contrario, se obtiene el ángulo reconstruido.

```
class Reconstruccion:

    def __init__(self, sep_cm: float):
        self.d = sep_cm

    def angle(self, dt_ns: float) -> Optional[float]:

        if dt_ns <= 0.0:
            return None

        cos_t = self.d / (C_LIGHT * dt_ns)

        if not (0.0 < cos_t <= 1.0):
            return None

        return float(np.arccos(cos_t))
```

3. Resultados

A continuación se presentan resultados para la simulación, generando 10000 muones y una separación de los centelladores de 20 cm.

3.1. Distribución angular generada y reconstruida

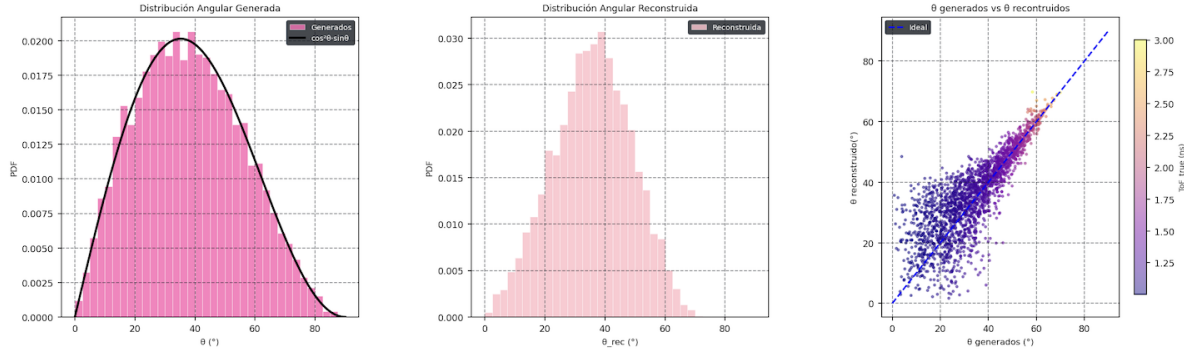


Figura 2: El primer gráfico (izquierda) muestra que los datos generados siguen la función teórica de distribución diferencial en ángulo sólido, considerando $\frac{dN}{d\Omega} \propto \cos^2 \theta$ el uso de esta función específica en el eje Y (densidad de probabilidad) permite normalizar los datos para que el área bajo la curva sea 1. El segundo gráfico (centro) presenta la distribución de los ángulos tras pasar por el proceso de detección y reconstrucción, y muestra cómo las limitaciones físicas del detector (como el jitter temporal y la saturación) alteran la forma original de los datos. Finalmente, el tercer gráfico (derecha) utiliza un diagrama de dispersión para correlacionar los ángulos generados con los reconstruidos, donde la línea diagonal representa el caso ideal y el código de color basado en el tiempo de vuelo (ToF) evidencia que la resolución angular mejora conforme aumenta el tiempo de interacción.

3.2. ToF

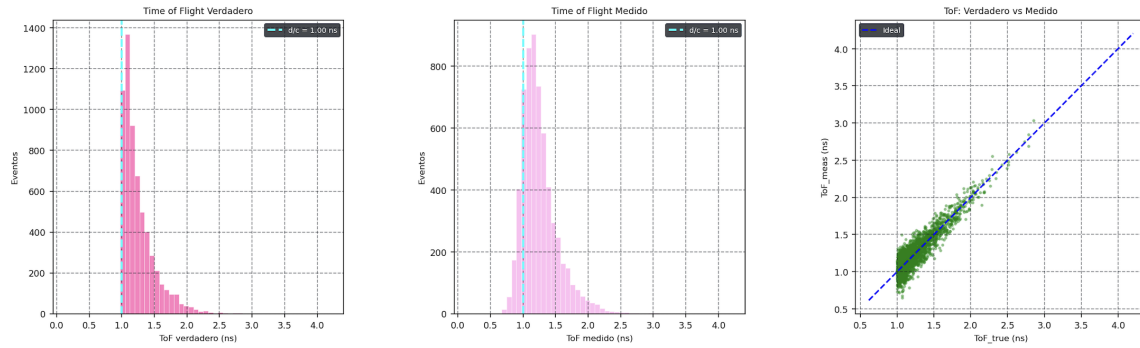


Figura 3: (izquierda) presenta el ToF verdadero la cual exhibe un límite inferior por $d/c = 1,00$ ns . (centro): muestra el ToF medido, el cual difiere respecto a al verdadero debido a factores estocásticos como el Transit Time Spread (TTS). (derecha) : diagrama de correlación entre ambos valores, donde la desviación de los datos respecto a la diagonal (línea ideal) constituye la incertidumbre temporal del sistema; la cuantificación de esta dispersión, expresada mediante la distribución de los residuos ($\Delta ToF = ToF_{med} - ToF_{true}$), define la resolución temporal efectiva del instrumento.

3.2.1. Residuo ToF

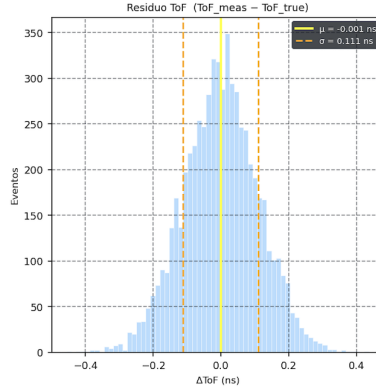


Figura 4: distribución de los residuos del tiempo de vuelo ($\Delta\text{ToF} = \text{ToF}_{\text{medido}} - \text{ToF}_{\text{verdadero}}$). una medida directa del error y la precisión del sistema de detección con media $\mu = -0,001$ ns y $\sigma = 0,111$ ns, la resolución temporal efectiva del sistema. Representa el ruido intrínseco o la incertidumbre que se añade a cualquier medición.

3.3. Energía depositada y número de fotoelectrones

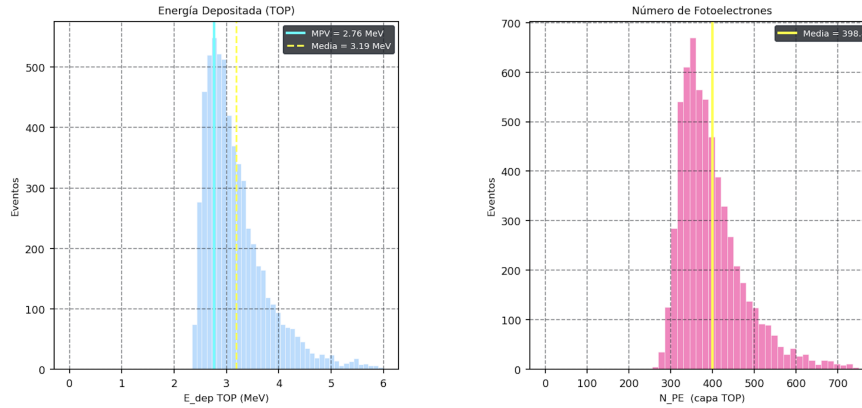


Figura 5: (izquierda): Energía depositada en capa superior caracterizado por un valor más probable de 2,76 MeV, (derecha) : número de fotoelectrones generados debido a la energía depositada en capa superior con media de 398,8.

3.4. Waveforms

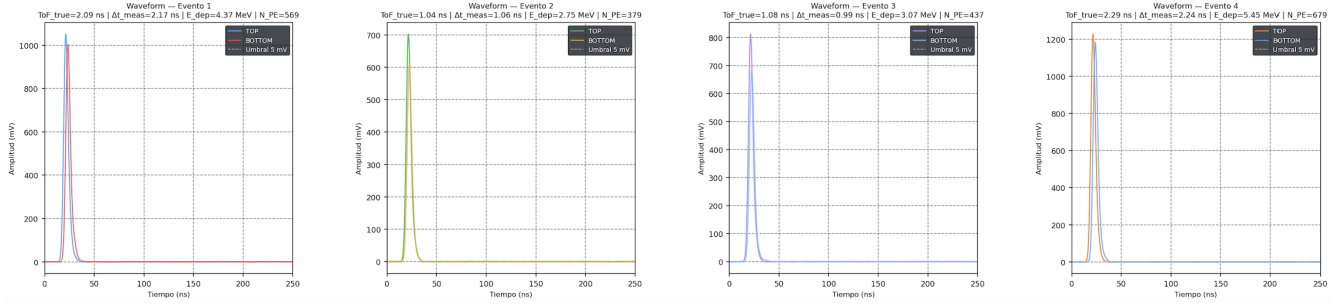


Figura 6: SE presenta las formas de onda simuladas para cuatro eventos individuales, ilustrando el proceso de adquisición y reconstrucción temporal. Cada panel superpone las señales eléctricas generadas en las capas superior (TOP) e inferior (BOTTOM) del detector. El instante de detección se define mediante un discriminador de umbral constante fijado en 5 mV. Los encabezados de cada evento demuestran la correlación directa entre la deposición de energía (E_{dep}), el rendimiento de fotoelectrones (N_{PE}) y la amplitud del pulso resultante. Asimismo, se evidencia la discrepancia entre el tiempo de vuelo verdadero (ToF_{true}) y el medido (Δt_{meas}).

3.5. Flujo relativo de muones versus altitud

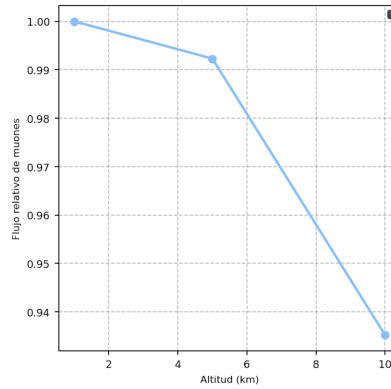


Figura 7: Fracción de supervivencia de muones atmosféricos en función de la profundidad atmosférica. El flujo está normalizado como $N(h)/N_0$, donde N_0 es el número de muones generados en la cima de la atmósfera, se calculó el número de muones a 1,5 y 10 km desde el punto de generación.

Referencias

- [1] Epic Crystal Co., Ltd., *BC-408 Plastic Scintillator*, https://www.epic-scintillator.com/Plastic-scintillator?product_id=1154.
- [2] G. F. Knoll, *Radiation Detection and Measurement*, 4th ed., Wiley (2010).
- [3] T. K. Gaisser, *Cosmic Rays and Particle Physics*, Cambridge University Press (1990).